

Scorm: Introduction to Programming Language and Compiler

Teacher: MEng. Tran Ngoc Bao Duy

I. Programming Languages

- Là các kí hiệu để mô tả tính toán của con người cho máy tính. Vì bản thân máy tính cần một trung gian để chuyển từ ngôn ngữ con người sang mã máy.
- **Là một bộ quy tắc** từ ngôn ngữ tự nhiên chuyển thành mã nhị phân (machine code), từ mã nhị phân máy tính sẽ thực hiện.
- Một số ngôn ngữ lập trình phổ biến:
 - Khoa học: Julia, MATLAB
 - Kinh tế: COBOL (B for Business)
 - AI: LISP, Prolog ... (khởi tạo từ logic vị từ)
 - Lập trình hệ thống: ngôn ngữ C
 - Lập trình web: XHTML, JS, PHP, ...
 - Ngôn ngữ đa chức năng: Python, C#, C++, sử dụng trên nhiều lĩnh vực khác nhau.

Language Evaluation

Tính chất cần thiết của một ngôn ngữ:

- có 4 tính chất của một ngôn ngữ cần có:
 - **Tính đơn giản:**
 - Một ngôn ngữ nên có cú pháp đơn giản.
 - Người lập trình chỉ cần suy nghĩ **làm sao để giải được bài toán**, và ngôn ngữ lập trình giúp người đó hiện thực hóa ý tưởng của bản thân → Đó là ngôn ngữ lập trình đơn giản
 - **Tính trực giao:**
 - **Những chức năng độc lập khác nhau trong ngôn ngữ khi kết hợp lại với nhau đều mang một ý nghĩa nào đó.**
 - Ví dụ: trong ngôn ngữ C ta khai báo `int x` → **Sự kết hợp giữa tên và kiểu dữ liệu (2 features độc lập với nhau) dùng để khai báo biến (meaningful) → có tính trực giao**
 - Exception (Ngoại lệ): Nếu ngoại lệ sinh ra càng nhiều → Ngôn ngữ càng ít trực giao. Ví dụ, ngôn ngữ C ở phần khai báo biến có một kiểu dữ liệu là `void` → Không được dùng

như kiểu dữ liệu cho tên để khai báo biến → Là ngoại lệ. **Vậy, C là một ngôn ngữ ít trực giao.**

- Một số ngôn ngữ trực giao: Haskell, Scheme
- Thiếu trực giao: **C, Java.**
- **Support of abstraction (tính hỗ trợ trừu tượng hóa):** Cần có sự hỗ trợ mạnh mẽ về trừu tượng hóa
 - Control Abstraction (trừu tượng hóa về mặt điều khiển): sử dụng các từ khóa thay cho một cơ chế nào đó: vòng lặp, rẽ nhánh, bất đồng bộ...
Ví dụ: for, while, async, if, switch...
 - Data Abstraction (trừu tượng hóa về mặt dữ liệu): **Một NNLT cần có đủ cơ chế để ta biểu diễn mọi loại dữ liệu mà chúng ta mong muốn.**
Ví dụ: C cung cấp struct để ta biểu diễn linked list. Các ngôn ngữ thường cung cấp cho chúng ta vòng lặp để tận dụng được khả năng của máy tính
 - Một cách dễ hiểu:
 - Trừu tượng hóa về mặt điều khiển → Liên quan tới flow code, những gì giúp lập trình viên hiện thực hóa ý tưởng mà không phải khai báo từng chút một như **vòng lặp, rẽ nhánh, đồng bộ ...**
 - Trừu tượng hóa về mặt dữ liệu → Liên quan tới kiểu dữ liệu. NNLT phải cho phép dev tự định nghĩa một kiểu dữ liệu phức tạp mới, thay vì chỉ được sử dụng các kiểu dữ liệu nguyên thủy. **Ví dụ: struct, class ...**
- **Safety (tính an toàn):** Ngôn ngữ cần cung cấp cơ chế để ta phát hiện lỗi của đoạn mã mà con người chúng ta viết ra. Tốt nhất là ngay lúc biên dịch (compile) hoặc chạy (runtime).

Developer cần một ngôn ngữ như thế nào?

- **Dễ đọc (readability).**
 - Lấy ví dụ, C có thể viết mọi lệnh lên cùng 1 dòng (miễn là có dấu chấm phẩy), vẫn có thể chạy được → Không đảm bảo thẩm mỹ vì người ngoài nhìn vào rất khó đọc.
 - Python thì ngược lại. Mọi câu lệnh phải theo quy tắc: Xuống dòng, thụt vào một tab → Đảm bảo cấu trúc dễ đọc cho tất cả mọi người.
- **Dễ dàng viết (writability).** Là mệnh đề trái ngược với dễ đọc. Một ngôn ngữ dễ đọc thường khó viết (vì phải trình bày), và viết như thế nào cũng được (dễ viết) thì thường khó đọc.
 - Với ngôn ngữ C, ta có thể viết như thế nào cũng được, code đều có thể chạy và biên dịch thành công. Tuy nhiên, output nếu viết ẩu ra cái gì thì ta rất khó kiểm soát.
 - Ngược lại, với Python, ta phải tuân theo cấu trúc và quy tắc của ngôn ngữ thì trình biên dịch mới hoạt động.
- **Dễ tin tưởng (reliability).** Cái này đi chung với tính **Safety** của ngôn ngữ. Một ngôn ngữ càng có các cơ chế safety tốt thì càng dễ tin tưởng.
 - Với ngôn ngữ C, nếu ta khai báo mảng tĩnh `a[5]` và tìm tới phần tử thứ 100: `a[100]` → trình biên dịch vẫn chạy, vẫn làm việc, dù chưa được khai báo tới đó → Không an toàn

- Với các ngôn ngữ bậc cao hơn thì luôn có cơ chế kiểm soát (cực đoan hơn, như trong Java, phải cần bỏ vào một bộ try ... except ...) cho bộ mã để gây ra lỗi thì chương trình mới chịu hoạt động.
- **Chi phí chấp nhận được (cost)**. Có 2 loại chi phí thường hay nói tới cho một NNLT: chi phí về con người và chi phí về thời gian.
 - Về con người, chi phí ở đây là chi phí chuyển từ một ngôn ngữ này sang một ngôn ngữ khác (cần thời gian học)
 - Về thời gian, chi phí ở đây là thời gian chạy (và cả biên dịch) của các ngôn ngữ. Nếu ngôn ngữ A chạy một chương trình nhanh hơn ngôn ngữ $B \rightarrow$ ngôn ngữ A tốn ít chi phí thời gian hơn ngôn ngữ B .

Influences on Languages Design (ảnh hưởng khi thiết kế NNLT)

- Ảnh hưởng về mặt kiến trúc máy tính:
 - **Đa số sử dụng kiến trúc Von Neumann**, gồm 3 bước:
 - Lấy từng lệnh **TUẦN TỰ** (fetch data)
 - Giải mã (decode)
 - Thực thi (execute)
- Ảnh hưởng về mặt phương pháp lập trình (programming methodologies). Có 2 phương pháp lập trình chính:
 - **Declarative (lập trình mang tính chất khai báo)** $\rightarrow$ Người lập trình cần cho máy tính biết là máy tính cần giải quyết điều gì. Nổi tiếng nhất là **Functional Programming** (Haskell, ML) hoặc **ngôn ngữ lập trình truy vấn có cấu trúc (SQL/GraphQL)**
 - **Imperative (lập trình mang tính tuần tự)**: Phải nói cho máy tính biết làm sao để giải quyết được bài toán đó. Hầu hết các ngôn ngữ hiện nay được xem là imperative. Các loại imperative thường gặp
 - **Lập trình hướng thủ tục** (Procedural-based): Fortran, ALGOL, COBOL, Pascal, C. Hầu hết các ngôn ngữ hiện tại là lập trình hướng thủ tục
 - **Lập trình hướng đối tượng** (Object-Oriented Programming)

Language Design Trade-off

1. Đánh đổi giữa độ tin cậy và chi phí thực thi
2. Đánh đổi giữa tính dễ đọc và tính dễ viết (đã nêu)
3. Đánh đổi về tính dễ viết và độ tin cậy (pointer trong C++ viết kiểu gì cũng được) $\rightarrow$ Không thể tin cậy C++ vì pointer có thể chỉ lung tung

II. Language Implementation



- Các phương pháp hiện thực ngôn ngữ bao gồm:

- Biên dịch
- Thông dịch
- Phương pháp lai
- Trình dịch tức thời

- **Khái niệm translators (chương trình dịch):**

- Source Code là một bộ quy tắc xây dựng từ ngôn ngữ tự nhiên, do con người tạo ra và là bước đầu tiên trong quy trình thực hiện giao tiếp giữa con người và máy tính.
- Machine Code là đoạn mã chỉ gồm các con số 0 và 1, máy tính có thể trực tiếp xử lý ở dạng này.
- **Đứng ở giữa Source code và machine code là một hệ thống dịch, với nhiệm vụ dịch từ source code ra machine code,** hệ thống đó gọi là chương trình dịch.

- *Một số khái niệm trong biểu đồ ở phần sau:*

- Compiler: Trình biên dịch.
- Interpreter: Trình thông dịch.
- Source Program: Source code, được viết bởi các ngôn ngữ do con người tạo ra.
- Target Program: thường là Machine Code - mã máy, gồm các con số 0 và 1.
- Immediate Program: Dịch tạm thời (gần giống với biên dịch, nhưng không biên dịch thành mã máy mà thường thành bytecode), thường được dùng trong hybrid system.
- Virtual Machine: Bản chất nó tương tự với interpreter
- Input: Đầu vào (số, chữ ...)
- Output: Kết quả trả về.

- Các loại chương trình dịch:

- **Biên dịch (compilation):** Source code sẽ được dịch trực tiếp thành mã nhị phân thông qua trình biên dịch (compiler). Mã nhị phân này làm việc trực tiếp với input để cho ra output.

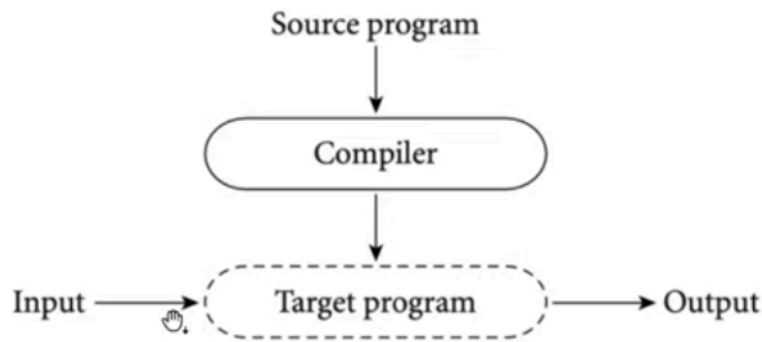


Figure: A compiler

Sơ đồ trình biên dịch

- **Thông dịch (pure interpreter):** input và source code cùng được đưa vào trình thông dịch (interpreter) một cách tuần tự. Tức nghĩa, mỗi dòng source code sẽ được thông dịch và trả về output ngay lập tức của dòng đó.

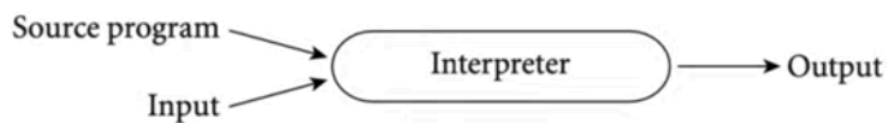


Figure: A pure interpreter

Sơ đồ trình thông dịch

- **So sánh compilation và interpreter:** Lai giữa thông dịch và biên dịch (compilation vs. pure interpreter). Làm như vậy để phát huy điểm mạnh và khắc phục điểm yếu của 2 phương pháp hiện thực ngôn ngữ trước.
 - **compilation dịch toàn bộ source code ra thành mã máy**, sau đó mới thực hiện. Trong **điều kiện rẽ nhánh (braching condition) như if-else**, compilation thể hiện **điểm yếu** khi dịch tất cả mã trong if-else, dù về mặt logic code thì người viết chỉ muốn lấy 1 trong các nhánh.
 - **interpreter dịch từng dòng code và thực hiện ngay lập tức**, nên hầu như luôn dịch lâu hơn compiler (interpreter phải dịch n lần thay vì 1 lần duy nhất). **Điểm yếu** này thể hiện rõ hơn khi vào **các vòng lặp (for-while)**, số vòng lặp càng nhiều thì lượng “dòng code” mà interpreter cần dịch càng tăng lên, khiến cho thời gian chạy không được tối ưu.

- Ngoài ra, **interpreter sẽ chuẩn đoán lỗi tốt hơn compiler**, nếu gặp lỗi ở dòng nào thì interpreter sẽ ngừng thông dịch luôn tại đó. Với compiler, việc dịch tất cả và chạy luôn cùng một lúc sẽ khiến nhiều lỗi ẩn xuất hiện không thể kiểm soát được.
- **Hệ lai (hybrid system)**: Lai giữa thông dịch và biên dịch (compilation vs. pure interpreter). Làm như vậy để phát huy điểm mạnh và khắc phục điểm yếu của 2 phương pháp hiện thực ngôn ngữ trước. Cụ thể:
 - Source code **trước tiên sẽ được biên dịch thành bytecode**, từ đó kết hợp với input, **trình thông dịch (trong hybrid gọi là virtual machine) sẽ dịch tuần tự từng dòng bytecode** để biến thành mã máy và trả về ngay lập tức kết quả.
 - **Virtual machine thường được biết đến nhất là JVM**, máy ảo của Java.
 - Điểm mạnh của Hybrid System: **Sau khi biên dịch source code thành bytecode, ta có thể đem bytecode đó tới mọi hệ điều hành (linux, window, MacOS,...) để tiếp tục chạy**, chỉ cần hệ điều hành đó có cài đặt Virtual Machine của ngôn ngữ. Case study nổi tiếng nhất về mặt này của Hybrid System là Java.

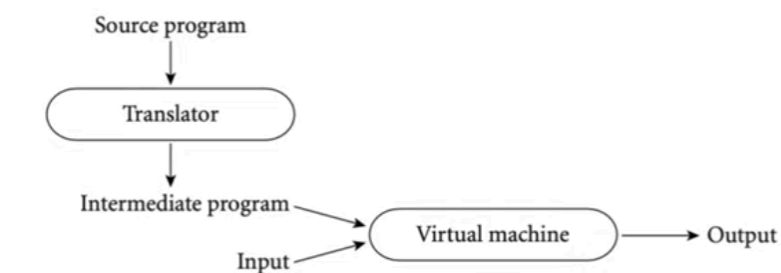
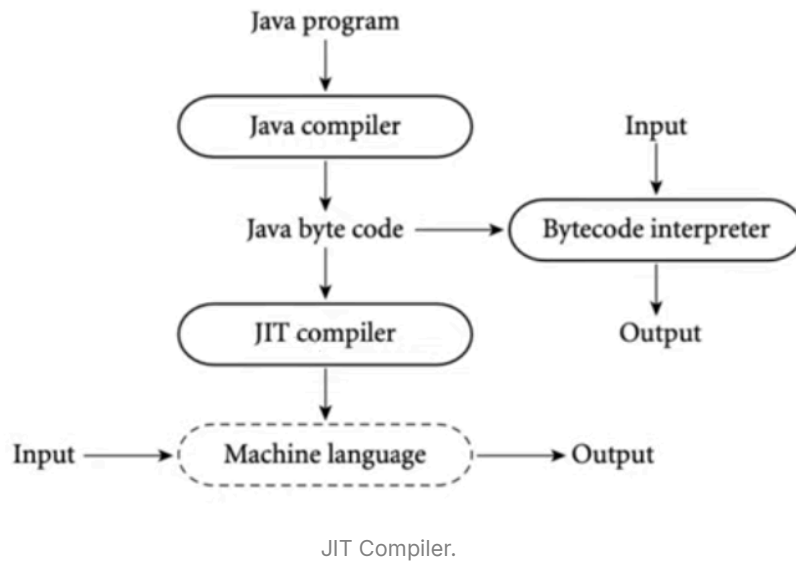


Figure: A hybrid system

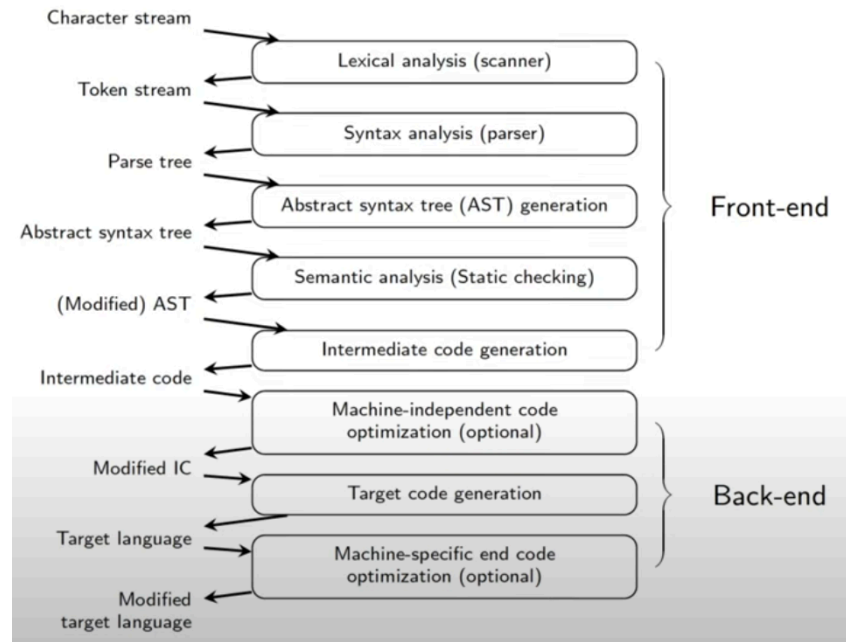
Hybrid system

- **Just-in-time (JIT) compilers**: Là một cải tiến của Hybrid System. Just-in-time (JIT) có nhiệm vụ **giải quyết điểm yếu về vòng lặp** của phần thông dịch (interpreter) trong hybrid.
 - Về mặt cấu trúc, JIT cơ bản giữ nguyên cấu trúc từ source code → compile thành bytecode → interpreter từng dòng để trả về kết quả. Tuy nhiên, có một sự can thiệp trong quá trình từ bytecode → interpreter.
 - **Hot method được định nghĩa là các đoạn code được phát hiện là lặp đi lặp lại**. Khi phát hiện ra trong bytecode có hot method, hệ thống sẽ **switch quá trình thông dịch** bytecode qua interpreter **thành quá trình biên dịch** bằng JIT compiler, nơi vòng lặp sẽ được biên dịch trực tiếp thành mã máy (machine code), và chạy vòng lặp trên machine code đó.
 - Kết thúc quá trình biên dịch vòng lặp trên JIT compiler, hệ thống lại tiếp tục thông dịch bằng interpreter ở các dòng tiếp theo (cho tới khi phát hiện lại hot method khác).



III. The structure of a compiler

- **Ban đầu ta có một mã nguồn - file (.cpp, .py, .c ...) về bản chất nó chỉ là file text - chuỗi các kí tự.** Ta sẽ học cách chuyển từ chuỗi kí tự này thành intermediate code.
- Trong môn học này, ta sẽ học cách xây dựng một compiler của một ngôn ngữ lập trình đơn giản. Đây là quá trình tổng quan để xây dựng một ngôn ngữ lập trình:
 - **Frontend**
 - Lexical Analysis (scanner)
 - Syntax Analysis (parser)
 - Abstract syntax tree (AST) Generation
 - Semantic Analysis (static checking)
 - Intermediate Code generation
 - **Backend**
 - Machine-independent code optimization
 - Target code generation
 - Machine-specific end code optimization (optional)
- Trong khuôn khổ môn học, ta focus vào phần front-end, tức dừng ở tạo intermediate code.



Overview of compilation

- **Quá trình phân tích từ vựng (lexical analysis) là quá trình đầu tiên.**

- Nhiệm vụ chính của lexical analysis là: từ **source code (chuỗi kí tự)**, ta **nhóm các kí tự** lại thành những **nhóm có nghĩa - được gọi là từ**. Sau đó, **phân loại những từ này vào trong các loại khác nhau, gọi là từ loại (token - lexemes)**. Các từ loại có thể là:

- **Danh hiệu (identifier - id):** tên gọi cho các biến
- **Kiểu:** data type
- **Dấu:** phép gán, cộng từ nhân chia ...

- Ngoài ra, còn có một vài chức năng khác:

- **Xóa các kí tự khoảng trắng** (space \s, tab \t, newline \n ...)
- **Xóa các comment/tags** - những thứ không có ý nghĩa đối với thực thi code.

- **Case study - Lexical Analysis:** Giả sử ta có lệnh trong C language: `a = b + c * 60;`, ta sẽ tách được ra thành **8 token** có dạng (Từ loại, chuỗi) như sau:

- 1. (ID, "a")
- 2. (EQUAL, "=")
- 3. (ID, "b")
- 4. (PLUS, "+")
- 5. (ID, "c")
- 6. (MUL, "*")

- 7. (INT, "60")
- 8. (SEMI, ";")



- Tổng kết, Thông qua quá trình Lexical Analysis), Ta biến đổi source từ một chuỗi kí tự thành chuỗi từ loại (token). Chuỗi từ loại này sẽ là input cho quá trình thứ hai - syntax analysis.

- **Quá trình phân tích ngữ pháp (syntax analysis) là quá trình thứ hai.**
 - Nhiệm vụ chính của quá trình này là **chuyển các token được sinh ra từ bước lexical analysis thành một parse tree** để mô tả cấu trúc ngữ pháp của bộ token (cách mà các token được sinh ra từ các quy tắc ngữ pháp).
 - Ví dụ, từ bộ token sinh từ dòng lệnh `a = b + c*60;` ở trên, ta tạo được parse tree như sau:

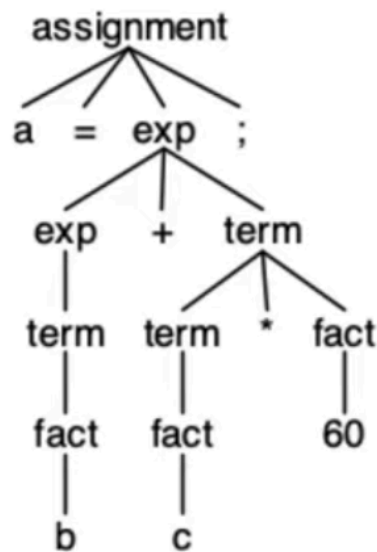
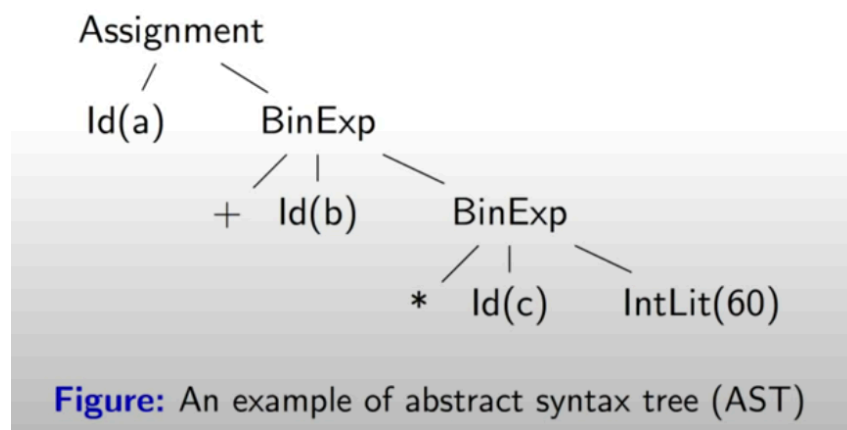


Figure: An example of parse tree

Parse tree generated from token stream

- **Quá trình sinh cây cú pháp trừu tượng (generated Abstract Syntax Tree (AST Generation)) là quá trình thứ ba.**
 - Một số compiler không trải qua bước này, sử dụng luôn parse tree để thực hiện bước tiếp theo.

- Nhiệm vụ chính của quá trình này là **loại bỏ các node (các lexeme/từ vựng) do con người quy định/tạo ra ("artificial" node)**.
 - Ví dụ, **Trong cây parse tree phía trên, có dấu "=" có ý nghĩa gán trong C - mang tính chất biểu tượng cho ngôn ngữ này.** Tuy nhiên, trong ngôn ngữ khác như Pascal, phép gán được biểu thị bởi dấu ":=". Về mặt ngữ nghĩa, sau khi sinh được parse tree tức là parse tree đã phù hợp, đầy đủ và chính xác. Vì vậy, việc giữ lại dấu "=" hay ":= " cho phép gán sẽ không có ý nghĩa. Ta loại bỏ dấu "=" trong AST Tree.
- Ví dụ với Parse Tree ở trên, ta loại bỏ đi các node chứa dấu đặc trưng của ngôn ngữ như dấu bằng, dấu chấm phẩy... để tạo nên AST Tree:



AST Tree generated from Parse Tree

- **Quá trình kiểm tra tĩnh - phân tích ngữ nghĩa (Semantic Analysis - Static Checking) là quá trình thứ tư.**
 - Nhiệm vụ chính của quá trình này là kiểm tra những rules/logic cần thiết, thường đặt câu hỏi "liệu nó có phù hợp hay không?". Việc kiểm tra này nhằm bổ sung, hoàn thiện cho những rules về ngữ pháp chính quy đã được mô tả.
 - Có 2 loại kiểm Semantic checking thường hay dùng:
 - **Kiểm tra ngữ pháp (syntactic checking):** Ngoài những ngữ pháp được mô tả chính quy, một số ràng buộc khác sẽ được kiểm tra trong giai đoạn này. Ví dụ về đặt tên, ta sẽ kiểm tra: "Tên đã được khai báo trước khi sử dụng chưa?"
 - **Kiểm tra về kiểu (type checking):** toán tử đó có phù hợp hay không. Ví dụ: "Dấu "+" có được sử dụng trong các biến string hay không?" sẽ được kiểm tra trong giai đoạn này.



- Lưu ý: Chữ "tĩnh" trong kiểm tra tĩnh có ý nghĩa là: Mọi kiểm tra phải bắt đầu trước khi dòng code đầu tiên chạy.

- **Quá trình hiện thực mã trung gian bytecode từ AST Tree (Intermediate Code generation) là quá trình cuối cùng của front-end.**
 - Trong môn học này, ta học về JVM - sinh ra Java Bytecode.
 - Ngoài ra còn một số loại Bytecode khác như CIL trong C#, CPython của Python...
- **Quá trình back-end chủ yếu là quá trình code optimization từ intermediate code. Mã trung gian sau khi tối ưu (tối ưu những bước không cần thiết) sẽ đưa vào quá trình sinh mã máy (machine code)**

Ứng dụng của compiler:

1. Cài đặt các ngôn ngữ bậc cao
2. Tối ưu hóa kiến trúc
3. Thiết kế những kiến trúc mới
4. Dịch các chương trình
5. Phát triển các công cụ phần mềm

Các chương trình có liên quan:

Preprocessor (trình tiền xử lý): xử lý dữ liệu đầu vào để tạo ra đầu ra dùng làm đầu vào cho một chương trình khác.

Assembler (trình hợp ngữ): chuyển các lệnh máy tính (mã hợp ngữ) thành mã máy.

Linker (trình liên kết): nhận một hoặc nhiều tệp đối tượng (được tạo bởi trình biên dịch hoặc trình hợp ngữ) và kết hợp chúng thành một tệp (thường là tệp thực thi hoặc thư viện).

Loader (trình nạp): thành phần chính của hệ điều hành, bảo đảm mọi chương trình và thư viện cần thiết được nạp vào bộ nhớ để thực thi.

Debugger (trình gỡ lỗi): dùng để kiểm thử và gỡ lỗi các chương trình khác.

Editor (trình soạn thảo): cho phép người dùng tạo và chỉnh sửa các tệp (mã nguồn).

IV. Quiz

1. **C++ có kiểu con trỏ rất mạnh. Kiểu này rất tốt cho người lập trình trong việc tùy biến cách viết, nhưng lại không tốt về?**
 - a. Chi phí (cost)
 - b. Tính tin cậy (Reliability)
 - c. Tính dễ viết (Wriability)
 - d. Tính dễ đọc (Readability)
2. **Khi một đoạn mã được đánh dấu là hot method, phương pháp hiện thực ngôn ngữ nào sẽ được sử dụng?**

- a. Trình thông dịch (pure interpreter)
 - b. Trình biên dịch (compilation)
 - c. Phương pháp lai (hybrid system)
 - d. Just-in-time Compiler (JIT)
3. **Nối các nội dung phù hợp với tên của giai đoạn trong việc thực hiện trình biên dịch?**
- a. Phân tích từ vựng
 - b. Phân tích ngữ pháp
 - c. Sinh cây cú pháp trừu tượng
1. Nhóm và phân loại các từ từ mã nguồn
 2. Loại bỏ những nút "nhân tạo" để tạo AST
 3. Tìm ngữ pháp phù hợp với chuỗi từ loại nhập

Đáp án câu 3: 1-a, 2-c, 3-b

4. **Giả sử một chương trình đơn giản được viết bởi ngôn ngữ X như sau:**
- Tính toán và trả về kết quả của biến A
 - In A ra màn hình
 - Tính toán một vài thứ khác nhưng hệ thống báo lỗi cú pháp (syntax error)
 - Chương trình được dịch và thực thi nhưng không có gì được in ra ngoại trừ thông báo lỗi cú pháp

Phương pháp dịch nào đã được sử dụng?

- a. Phương pháp lai (Hybrid Implementation)
 - b. Just-in-time (JIT) compilation
 - c. Trình biên dịch (Compilation)
 - d. Trình thông dịch (Pure interpretation)
5. **Một ngôn ngữ lập trình có nhiều cách để giúp nhà phát triển định nghĩa một kiểu mới để thể hiện một cấu trúc dữ liệu mới. Ngôn ngữ lập trình trên tốt về?**
- a. **Support of data abstraction**
 - b. Orthogonality
 - c. Support of control abstraction
 - d. Safety
6. **Phương pháp hiện thực nào chỉ hoạt động khi một số method được phát hiện là hot method?**
- a. Pure interpretation
 - b. **Just-in-time (JIT) Compilation**
 - c. Hybrid implementation

- d. Compilation
- 7. Một ngôn ngữ lập trình khai báo (declarative programming language) yêu cầu người phát triển phải viết mã cho máy tính biết?
 - a. **Vấn đề là gì**
 - b. cách giải quyết vấn đề như thế nào
 - c. Khi nào vấn đề xảy ra
 - d. Vấn đề xảy ra tại đâu
- 8. Một ngôn ngữ lập trình có nhiều cơ chế để giúp người phát triển phân tích những trình tự thực thi khác nhau. NNLT đó tốt về?
 - a. **Support of control abstraction**
 - b. Support of data abstraction
 - c. Safety
 - d. Tính trực giao (Orthogonality)
- 9. Một NNLT tuần tự (imperative programming language) yêu cầu người dùng phải viết cái gì?
 - a. Khi nào vấn đề xảy ra
 - b. Định nghĩa vấn đề là gì
 - c. **Cách để giải quyết vấn đề**
 - d. Vấn đề xảy ra ở đâu
- 10. Ngôn ngữ Java thực hiện kiểm tra động (dynamic checking) cho tất cả tham chiếu đến các phần tử trong mảng để chắc chắn không có tham chiếu nào trỏ tới phần tử ngoài mảng. Tính năng này chứng tỏ Java tốt về.... nhưng tệ về ?

→ Trả lời: tốt về Reliability, tệ về Cost
- 11. Một NNLT có thể phát hiện ra nhiều lỗi tĩnh. NNLT đó tốt về?
 - a. Support of data abstraction
 - b. Support of control abstraction
 - c. Orthogonality
 - d. **Safety**
- 12. Một chương trình chạy một chương trình thực thi từ bộ nhớ phụ sang bộ nhớ chính và sắp xếp cho mỗi lần chạy. Chương trình đó là?
 - a. Assembler (hợp ngữ)
 - b. **Loader (trình nạp)**
 - c. Link editor (trình liên kết)
 - d. Debugger (trình gỡ lỗi)